

por favor, crie o código que já faça ligação com um banco de dados sql server e mande pra mim também os códigos das tabelas

Vamos construir uma solução completa: frontend HTML + JavaScript (com chamadas fetch para uma API) e backend em Node.js + Express + mssql (para SQL Server). Vou fornecer todos os códigos necessários, incluindo:

1. **Script SQL** para criar as tabelas no SQL Server.
2. **Backend (Node.js + Express)** com rotas para produtos, usuários, autenticação, carrinho, reservas, etc.
3. **Frontend adaptado** para consumir a API, substituindo o localStorage por chamadas fetch.

Como o projeto é extenso, vou organizar em partes e explicar cada uma. O foco é fazer o sistema funcionar com SQL Server.

Scripts SQL (SQL Server)

Execute esses comandos no SQL Server Management Studio ou ferramenta similar para criar o banco e as tabelas.

```
sql

-- Cria o banco de dados (opcional, você pode usar um existente)
CREATE DATABASE SoulforeDB;
GO

USE SoulforeDB;
GO

-- Tabela de usuários (clientes e admins)
CREATE TABLE Usuarios (
    id INT IDENTITY(1,1) PRIMARY KEY,
    nome VARCHAR(150) NOT NULL,
    cpf CHAR(11) UNIQUE NOT NULL,
    telefone VARCHAR(20),
    email VARCHAR(100) UNIQUE NOT NULL,
    senha VARCHAR(255) NOT NULL, -- guardaremos hash (bcrypt)
    tipo VARCHAR(20) DEFAULT 'cliente', -- 'cliente' ou 'admin'
    data_cadastro DATETIME DEFAULT GETDATE()
);
GO
```

```

-- Tabela de produtos
CREATE TABLE Produtos (
    id INT IDENTITY(1,1) PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    categoria VARCHAR(50),
    tamanho VARCHAR(10),
    cor VARCHAR(30),
    preco DECIMAL(10,2) NOT NULL,
    estoque INT NOT NULL,
    descricao TEXT,
    imagem VARCHAR(500) -- URL da imagem
);
GO

-- Tabela de alugueis (pedidos)
CREATE TABLE Alugueis (
    id INT IDENTITY(1,1) PRIMARY KEY,
    id_cliente INT NOT NULL FOREIGN KEY REFERENCES Usuarios(id),
    data_retirada DATE NOT NULL,
    data_devolucao DATE NOT NULL,
    status VARCHAR(30) DEFAULT 'pendente', -- pendente, confirmado,
cancelado, concluido
    valor_total DECIMAL(10,2) NOT NULL,
    sinal_pago DECIMAL(10,2),
    data_criacao DATETIME DEFAULT GETDATE()
);
GO

-- Itens do aluguel (relação N:N entre aluguel e produto)
CREATE TABLE ItensAluguel (
    id INT IDENTITY(1,1) PRIMARY KEY,
    id_aluguel INT NOT NULL FOREIGN KEY REFERENCES Alugueis(id) ON DELETE
CASCADE,
    id_produto INT NOT NULL FOREIGN KEY REFERENCES Produtos(id),
    quantidade INT NOT NULL,
    valor_unitario DECIMAL(10,2) NOT NULL -- preço no momento da reserva
);
GO

-- Tabela de favoritos (relação usuário-produto)
CREATE TABLE Favoritos (
    id_usuario INT NOT NULL FOREIGN KEY REFERENCES Usuarios(id),
    id_produto INT NOT NULL FOREIGN KEY REFERENCES Produtos(id),

```

```
    PRIMARY KEY (id_usuario, id_produto)
);
GO

-- (Opcional) Tabela de tokens para refresh, etc.
```

⚙ Backend (Node.js + Express + mssql)

Estrutura de pastas sugerida:

```
text
backend/
├── .env
├── package.json
├── server.js
├── routes/
│   ├── produtos.js
│   ├── auth.js
│   ├── usuarios.js
│   ├── alugueis.js
│   └── favoritos.js
├── controllers/
├── db.js (conexão)
└── middleware/
```

1. Configuração inicial

package.json (dependências principais)

```
json
{
  "name": "soulfore-backend",
  "version": "1.0.0",
  "description": "API para sistema de aluguel",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "mssql": "^9.1.1",
    "dotenv": "^16.0.3",
```

```

    "bcrypt": "^5.1.0",
    "jsonwebtoken": "^9.0.0",
    "cors": "^2.8.5"
  },
  "devDependencies": {
    "nodemon": "^2.0.22"
  }
}

```

.env (configuração do banco e JWT)

```

text

DB_SERVER=localhost
DB_DATABASE=SoulforeDB
DB_USER=sa
DB_PASSWORD=sua_senha
JWT_SECRET=segredo_super_secreto
PORT=3000

```

2. Conexão com o banco (db.js)

```

javascript

const sql = require('mssql');
require('dotenv').config();

const config = {
  server: process.env.DB_SERVER,
  database: process.env.DB_DATABASE,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  options: {
    encrypt: false, // se estiver usando local, false; para Azure,
    true
    enableArithAbort: true,
    trustServerCertificate: true // para conexões locais
  }
};

let pool;

async function getConnection() {
  try {
    if (pool) {
      console.log('Usando conexão existente');
      return pool;
    }
  }
}

```

```

    }
    pool = await sql.connect(config);
    console.log('Conectado ao SQL Server');
    return pool;
  } catch (err) {
    console.error('Erro ao conectar ao banco:', err);
    throw err;
  }
}

module.exports = { getConnection, sql };

```

3. Servidor principal (server.js)

javascript

```

const express = require('express');
const cors = require('cors');
require('dotenv').config();
const { getConnection } = require('./db');

const app = express();
app.use(cors());
app.use(express.json());

// Rotas
const authRoutes = require('./routes/auth');
const produtosRoutes = require('./routes/produtos');
const usuariosRoutes = require('./routes/usuarios');
const alugueisRoutes = require('./routes/alugueis');
const favoritosRoutes = require('./routes/favoritos');

app.use('/api/auth', authRoutes);
app.use('/api/produtos', produtosRoutes);
app.use('/api/usuarios', usuariosRoutes);
app.use('/api/alugueis', alugueisRoutes);
app.use('/api/favoritos', favoritosRoutes);

// Teste de conexão
app.get('/api/test', async (req, res) => {
  try {
    await getConnection();
    res.send('Conectado ao banco!');
  } catch (err) {
    res.status(500).send('Erro de conexão');
  }
});

```

```

    }
  });

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Servidor rodando na porta ${PORT}`));

```

4. Rota de autenticação (auth.js)

javascript

```

const express = require('express');
const router = express.Router();
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const { getConnection, sql } = require('../db');

// Registro de novo usuário
router.post('/register', async (req, res) => {
  const { nome, cpf, telefone, email, senha } = req.body;
  if (!nome || !cpf || !email || !senha) {
    return res.status(400).json({ error: 'Campos obrigatórios faltando' });
  }
  try {
    const pool = await getConnection();
    // Verificar se usuário já existe
    const existing = await pool.request()
      .input('email', sql.VarChar, email)
      .input('cpf', sql.VarChar, cpf)
      .query('SELECT id FROM Usuarios WHERE email = @email OR cpf = @cpf');
    if (existing.recordset.length > 0) {
      return res.status(409).json({ error: 'E-mail ou CPF já cadastrado' });
    }
    // Hash da senha
    const hashedPassword = await bcrypt.hash(senha, 10);
    const result = await pool.request()
      .input('nome', sql.VarChar, nome)
      .input('cpf', sql.VarChar, cpf)
      .input('telefone', sql.VarChar, telefone || null)
      .input('email', sql.VarChar, email)
      .input('senha', sql.VarChar, hashedPassword)
      .input('tipo', sql.VarChar, 'cliente')
      .query(`
        INSERT INTO Usuarios (nome, cpf, telefone, email, senha,

```

```

tipo)

        OUTPUT inserted.id
        VALUES (@nome, @cpf, @telefone, @email, @senha, @tipo)
    `);

    const userId = result.recordset[0].id;
    const token = jwt.sign({ id: userId, email, tipo: 'cliente' },
process.env.JWT_SECRET, { expiresIn: '7d' });
    res.status(201).json({ message: 'Usuário criado', token, user: {
id: userId, nome, email, tipo: 'cliente' } });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Erro no servidor' });
  }
});

// Login
router.post('/login', async (req, res) => {
  const { email, senha } = req.body;
  if (!email || !senha) {
    return res.status(400).json({ error: 'E-mail e senha
obrigatórios' });
  }
  try {
    const pool = await getConnection();
    const result = await pool.request()
      .input('email', sql.VarChar, email)
      .query('SELECT id, nome, email, senha, tipo FROM Usuarios
WHERE email = @email');
    if (result.recordset.length === 0) {
      return res.status(401).json({ error: 'Credenciais inválidas'
});
    }
    const user = result.recordset[0];
    const match = await bcrypt.compare(senha, user.senha);
    if (!match) {
      return res.status(401).json({ error: 'Credenciais inválidas'
});
    }
    const token = jwt.sign({ id: user.id, email: user.email, tipo:
user.tipo }, process.env.JWT_SECRET, { expiresIn: '7d' });
    res.json({ message: 'Login bem-sucedido', token, user: { id:
user.id, nome: user.nome, email: user.email, tipo: user.tipo } });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Erro no servidor' });
  }
});

```

```

    }
  });

module.exports = router;

```

5. Middleware de autenticação (authMiddleware.js)

javascript

```

const jwt = require('jsonwebtoken');

function authMiddleware(req, res, next) {
  const authHeader = req.headers.authorization;
  if (!authHeader) {
    return res.status(401).json({ error: 'Token não fornecido' });
  }
  const token = authHeader.split(' ')[1];
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Token inválido' });
  }
}

module.exports = authMiddleware;

```

6. Rota de produtos (produtos.js)

javascript

```

const express = require('express');
const router = express.Router();
const { getConnection, sql } = require('../db');
const authMiddleware = require('../middleware/authMiddleware');

// Listar todos os produtos (público)
router.get('/', async (req, res) => {
  try {
    const pool = await getConnection();
    const result = await pool.request().query('SELECT * FROM
Produtos');
    res.json(result.recordset);
  } catch (err) {
    console.error(err);
  }
});

```



```

        res.status(500).json({ error: 'Erro ao buscar produtos' });
    }
});

// Buscar produto por ID (público)
router.get('/:id', async (req, res) => {
    const { id } = req.params;
    try {
        const pool = await getConnection();
        const result = await pool.request()
            .input('id', sql.Int, id)
            .query('SELECT * FROM Produtos WHERE id = @id');
        if (result.recordset.length === 0) {
            return res.status(404).json({ error: 'Produto não encontrado' });
        }
        res.json(result.recordset[0]);
    } catch (err) {
        console.error(err);
        res.status(500).json({ error: 'Erro ao buscar produto' });
    }
});

// Criar produto (somente admin)
router.post('/', authMiddleware, async (req, res) => {
    if (req.user.tipo !== 'admin') {
        return res.status(403).json({ error: 'Acesso negado' });
    }
    const { nome, categoria, tamanho, cor, preco, estoque, descricao,
imagem } = req.body;
    if (!nome || !preco || !estoque) {
        return res.status(400).json({ error: 'Nome, preço e estoque são obrigatórios' });
    }
    try {
        const pool = await getConnection();
        const result = await pool.request()
            .input('nome', sql.VarChar, nome)
            .input('categoria', sql.VarChar, categoria || null)
            .input('tamanho', sql.VarChar, tamanho || null)
            .input('cor', sql.VarChar, cor || null)
            .input('preco', sql.Decimal, preco)
            .input('estoque', sql.Int, estoque)
            .input('descricao', sql.Text, descricao || null)
            .input('imagem', sql.VarChar, imagem || null)

```

```

        .query(`
            INSERT INTO Produtos (nome, categoria, tamanho, cor,
preco, estoque, descricao, imagem)
            OUTPUT inserted.id
            VALUES (@nome, @categoria, @tamanho, @cor, @preco,
@estoque, @descricao, @imagem)
        `);
        res.status(201).json({ message: 'Produto criado', id:
result.recordset[0].id });
    } catch (err) {
        console.error(err);
        res.status(500).json({ error: 'Erro ao criar produto' });
    }
});

// Atualizar produto (admin)
router.put('/:id', authMiddleware, async (req, res) => {
    if (req.user.tipo !== 'admin') {
        return res.status(403).json({ error: 'Acesso negado' });
    }
    const { id } = req.params;
    const { nome, categoria, tamanho, cor, preco, estoque, descricao,
imagem } = req.body;
    try {
        const pool = await getConnection();
        await pool.request()
            .input('id', sql.Int, id)
            .input('nome', sql.VarChar, nome)
            .input('categoria', sql.VarChar, categoria)
            .input('tamanho', sql.VarChar, tamanho)
            .input('cor', sql.VarChar, cor)
            .input('preco', sql.Decimal, preco)
            .input('estoque', sql.Int, estoque)
            .input('descricao', sql.Text, descricao)
            .input('imagem', sql.VarChar, imagem)
            .query(`
                UPDATE Produtos SET
                    nome = @nome,
                    categoria = @categoria,
                    tamanho = @tamanho,
                    cor = @cor,
                    preco = @preco,
                    estoque = @estoque,
                    descricao = @descricao,
                    imagem = @imagem
            `);
    } catch (err) {
        console.error(err);
        res.status(500).json({ error: 'Erro ao atualizar produto' });
    }
});

```

```

        WHERE id = @id
    `);
    res.json({ message: 'Produto atualizado' });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Erro ao atualizar produto' });
  }
});

// Deletar produto (admin)
router.delete('/:id', authMiddleware, async (req, res) => {
  if (req.user.tipo !== 'admin') {
    return res.status(403).json({ error: 'Acesso negado' });
  }
  const { id } = req.params;
  try {
    const pool = await getConnection();
    await pool.request()
      .input('id', sql.Int, id)
      .query('DELETE FROM Produtos WHERE id = @id');
    res.json({ message: 'Produto deletado' });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Erro ao deletar produto' });
  }
});

module.exports = router;

```

7. Rota de favoritos (favoritos.js)

javascript

```

const express = require('express');
const router = express.Router();
const { getConnection, sql } = require('../db');
const authMiddleware = require('../middleware/authMiddleware');

// Adicionar/remover favorito (toggle)
router.post('/toggle/:produtoId', authMiddleware, async (req, res) => {
  const usuarioId = req.user.id;
  const produtoId = req.params.produtoId;
  try {
    const pool = await getConnection();
    // Verificar se já existe
    const check = await pool.request()

```

```

        .input('usuarioId', sql.Int, usuarioId)
        .input('produtoId', sql.Int, produtoId)
        .query('SELECT * FROM Favoritos WHERE id_usuario = @usuarioId
AND id_produto = @produtoId');
        if (check.recordset.length > 0) {
            // Remover
            await pool.request()
                .input('usuarioId', sql.Int, usuarioId)
                .input('produtoId', sql.Int, produtoId)
                .query('DELETE FROM Favoritos WHERE id_usuario =
@usuarioId AND id_produto = @produtoId');
            res.json({ favorito: false });
        } else {
            // Adicionar
            await pool.request()
                .input('usuarioId', sql.Int, usuarioId)
                .input('produtoId', sql.Int, produtoId)
                .query('INSERT INTO Favoritos (id_usuario, id_produto)
VALUES (@usuarioId, @produtoId)');
            res.json({ favorito: true });
        }
    } catch (err) {
        console.error(err);
        res.status(500).json({ error: 'Erro ao alternar favorito' });
    }
});

// Listar favoritos do usuário
router.get('/', authMiddleware, async (req, res) => {
    const usuarioId = req.user.id;
    try {
        const pool = await getConnection();
        const result = await pool.request()
            .input('usuarioId', sql.Int, usuarioId)
            .query(`
                SELECT p.* FROM Produtos p
                INNER JOIN Favoritos f ON p.id = f.id_produto
                WHERE f.id_usuario = @usuarioId
            `);
        res.json(result.recordset);
    } catch (err) {
        console.error(err);
        res.status(500).json({ error: 'Erro ao buscar favoritos' });
    }
});

```

```
module.exports = router;
```

8. Rota de alugueis (alugueis.js) – simplificada

javascript

```
const express = require('express');
const router = express.Router();
const { getConnection, sql } = require('../db');
const authMiddleware = require('../middleware/authMiddleware');

// Criar um novo aluguel (a partir do carrinho)
router.post('/', authMiddleware, async (req, res) => {
  const usuarioId = req.user.id;
  const { itens, dataRetirada, dataDevolucao } = req.body; // itens:
  // array de {id_produto, quantidade}
  if (!itens || !dataRetirada || !dataDevolucao) {
    return res.status(400).json({ error: 'Dados incompletos' });
  }
  const pool = await getConnection();
  const transaction = new sql.Transaction(pool);
  try {
    await transaction.begin();
    // Calcular total e verificar estoque
    let valorTotal = 0;
    const itensComPreco = [];
    for (let item of itens) {
      const prod = await transaction.request()
        .input('id', sql.Int, item.id_produto)
        .query('SELECT preco, estoque FROM Produtos WHERE id =
@id');
      if (prod.recordset.length === 0) throw new Error('Produto não
encontrado');
      if (prod.recordset[0].estoque < item.quantidade) throw new
Error('Estoque insuficiente para o produto ' + item.id_produto);
      valorTotal += prod.recordset[0].preco * item.quantidade;
      itensComPreco.push({
        id_produto: item.id_produto,
        quantidade: item.quantidade,
        valor_unitario: prod.recordset[0].preco
      });
    }
    const sinal = valorTotal * 0.3; // RN01: sinal de 30%
    // Inserir aluguel
    const insertAluguel = await transaction.request()
```

```

        .input('id_cliente', sql.Int, usuarioId)
        .input('data_retirada', sql.Date, dataRetirada)
        .input('data_devolucao', sql.Date, dataDevolucao)
        .input('status', sql.VarChar, 'pendente')
        .input('valor_total', sql.Decimal, valorTotal)
        .input('sinal_pago', sql.Decimal, sinal)
        .query(`
            INSERT INTO Alugueis (id_cliente, data_retirada,
data_devolucao, status, valor_total, sinal_pago)
            OUTPUT inserted.id
            VALUES (@id_cliente, @data_retirada, @data_devolucao,
@status, @valor_total, @sinal_pago)
        `);
    const idAluguel = insertAluguel.recordset[0].id;
    // Inserir itens
    for (let item of itensComPreco) {
        await transaction.request()
            .input('id_aluguel', sql.Int, idAluguel)
            .input('id_produto', sql.Int, item.id_produto)
            .input('quantidade', sql.Int, item.quantidade)
            .input('valor_unitario', sql.Decimal,
item.valor_unitario)
            .query(`
                INSERT INTO ItensAluguel (id_aluguel, id_produto,
quantidade, valor_unitario)
                VALUES (@id_aluguel, @id_produto, @quantidade,
@valor_unitario)
            `);
        // Atualizar estoque (diminuir)
        await transaction.request()
            .input('id_produto', sql.Int, item.id_produto)
            .input('quantidade', sql.Int, item.quantidade)
            .query('UPDATE Produtos SET estoque = estoque -
@quantidade WHERE id = @id_produto');
    }
    await transaction.commit();
    res.status(201).json({ message: 'Aluguel criado com sucesso', id:
idAluguel, sinalPago: sinal });
    } catch (err) {
        await transaction.rollback();
        console.error(err);
        res.status(500).json({ error: err.message || 'Erro ao criar
aluguel' });
    }
});

```

```

// Listar alugueis do usuário
router.get('/meus', authMiddleware, async (req, res) => {
  const usuarioId = req.user.id;
  try {
    const pool = await getConnection();
    const result = await pool.request()
      .input('id_cliente', sql.Int, usuarioId)
      .query('SELECT * FROM Alugueis WHERE id_cliente = @id_cliente
ORDER BY data_criacao DESC');
    res.json(result.recordset);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Erro ao buscar alugueis' });
  }
});

module.exports = router;

```

9. Rota de usuários (para perfil)

```

javascript

const express = require('express');
const router = express.Router();
const { getConnection, sql } = require('../db');
const authMiddleware = require('../middleware/authMiddleware');

// Obter perfil do usuário Logado
router.get('/me', authMiddleware, async (req, res) => {
  try {
    const pool = await getConnection();
    const result = await pool.request()
      .input('id', sql.Int, req.user.id)
      .query('SELECT id, nome, cpf, telefone, email, tipo,
data_cadastro FROM Usuarios WHERE id = @id');
    if (result.recordset.length === 0) {
      return res.status(404).json({ error: 'Usuário não encontrado'
});
    }
    res.json(result.recordset[0]);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Erro ao buscar perfil' });
  }
});

```

```
// Atualizar perfil
router.put('/me', authMiddleware, async (req, res) => {
  const { nome, telefone } = req.body;
  try {
    const pool = await getConnection();
    await pool.request()
      .input('id', sql.Int, req.user.id)
      .input('nome', sql.VarChar, nome)
      .input('telefone', sql.VarChar, telefone)
      .query('UPDATE Usuarios SET nome = @nome, telefone =
@telefone WHERE id = @id');
    res.json({ message: 'Perfil atualizado' });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Erro ao atualizar perfil' });
  }
});

module.exports = router;
```

🌐 Frontend Adaptado para API

Vou fornecer uma versão simplificada do frontend, adaptando as funções que antes usavam `localStorage` para usar `fetch`. O código completo seria muito extenso, mas darei os principais trechos de modificação e um exemplo funcional com as principais funções.

Estrutura do frontend (HTML + JS)

Manteremos o mesmo HTML, mas substituiremos o bloco `<script>` por um novo que faz chamadas à API.

Principais mudanças:

- **Login:** agora envia requisição para `/api/auth/login` e armazena o token JWT no `localStorage`.
- **Registro:** envia para `/api/auth/register`.
- **Produtos:** busca da API (`/api/produtos`) e exibe.
- **Admin:** ao salvar produto, envia POST/PUT com token.
- **Carrinho:** continua no frontend (`localStorage`) até a finalização, quando então envia para `/api/alugueis`.

- **Favoritos:** usa a rota `/api/favoritos/toggle/:id`.

Vou fornecer um exemplo condensado com as principais funções.

javascript

```
// Configuração base da API
const API_URL = 'http://localhost:3000/api'; // ajuste para o IP/porta do backend

// Função para pegar token
function getToken() {
    return localStorage.getItem('token');
}

// Headers com autenticação
function authHeaders() {
    const token = getToken();
    return {
        'Content-Type': 'application/json',
        'Authorization': token ? `Bearer ${token}` : ''
    };
}

// ===== AUTENTICAÇÃO =====
async function fazerLogin() {
    const email = document.getElementById('loginEmail').value;
    const senha = document.getElementById('loginPassword').value;
    try {
        const response = await fetch(`${API_URL}/auth/login`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ email, senha })
        });
        const data = await response.json();
        if (!response.ok) throw new Error(data.error);
        localStorage.setItem('token', data.token);
        localStorage.setItem('user', JSON.stringify(data.user));
        updateMenuByUser();
        showView('account');
    } catch (error) {
        alert('Erro no login: ' + error.message);
    }
}
```

```

async function criarConta() {
  const nome = document.getElementById('regNome').value;
  const cpf = document.getElementById('regCpf').value;
  const tel = document.getElementById('regTel').value;
  const email = document.getElementById('regEmail').value;
  const senha = document.getElementById('regSenha').value;
  const conf = document.getElementById('regConfSenha').value;
  const termos = document.getElementById('regTermos').checked;
  if (senha !== conf) { alert('Senhas não conferem'); return; }
  try {
    const response = await fetch(`${API_URL}/auth/register`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ nome, cpf, telefone: tel, email, senha
    })

    });
    const data = await response.json();
    if (!response.ok) throw new Error(data.error);
    localStorage.setItem('token', data.token);
    localStorage.setItem('user', JSON.stringify(data.user));
    updateMenuByUser();
    showView('account');
  } catch (error) {
    alert('Erro no cadastro: ' + error.message);
  }
}

function logout() {
  localStorage.removeItem('token');
  localStorage.removeItem('user');
  updateMenuByUser();
  showView('home');
}

// ===== PRODUTOS =====
async function carregarProdutos() {
  try {
    const response = await fetch(`${API_URL}/produtos`);
    const produtos = await response.json();
    localStorage.setItem('produtosCache', JSON.stringify(produtos));
    // opcional
    renderProductList(produtos);
    renderHome(produtos);
  } catch (error) {
    console.error('Erro ao carregar produtos:', error);
  }
}

```

```

    }
}

async function salvarProduto() {
    const produto = {
        nome: document.getElementById('prodNome').value,
        categoria: document.getElementById('prodCategoria').value,
        tamanho: document.getElementById('prodTamanho').value,
        cor: document.getElementById('prodCor').value,
        preco: parseFloat(document.getElementById('prodPreco').value),
        estoque: parseInt(document.getElementById('prodEstoque').value),
        descricao: document.getElementById('prodDesc').value,
        imagem: document.getElementById('prodImagem').value
    };
    const id = document.getElementById('prodId').value;
    const method = id ? 'PUT' : 'POST';
    const url = id ? `${API_URL}/produtos/${id}` : `${API_URL}/produtos`;
    try {
        const response = await fetch(url, {
            method,
            headers: authHeaders(),
            body: JSON.stringify(produto)
        });
        if (!response.ok) throw new Error('Erro ao salvar');
        fecharModalProduto();
        carregarProdutos(); // recarrega lista
        renderAdminProducts(); // atualiza tabela admin
    } catch (error) {
        alert('Erro: ' + error.message);
    }
}

async function excluirProduto(id) {
    if (!confirm('Confirma exclusão?')) return;
    try {
        const response = await fetch(`${API_URL}/produtos/${id}`, {
            method: 'DELETE',
            headers: authHeaders()
        });
        if (!response.ok) throw new Error('Erro ao excluir');
        carregarProdutos();
        renderAdminProducts();
    } catch (error) {
        alert('Erro: ' + error.message);
    }
}

```

```

}

// ===== FAVORITOS =====
async function toggleFavorito(prodId, event) {
  event.stopPropagation();
  const token = getToken();
  if (!token) {
    alert('Faça login para favoritar');
    return;
  }
  try {
    const response = await
fetch(`${API_URL}/favoritos/toggle/${prodId}`, {
  method: 'POST',
  headers: authHeaders()
});
    const data = await response.json();
    // Atualizar ícone visual
    carregarProdutos(); // simplificado: recarrega tudo
  } catch (error) {
    console.error(error);
  }
}

// ===== FINALIZAR RESERVA =====
async function confirmarReserva() {
  const cart = JSON.parse(localStorage.getItem('cart')) || [];
  if (cart.length === 0) { alert('Carrinho vazio'); return; }
  const itens = cart.map(item => ({
    id_produto: item.id,
    quantidade: item.quantity
  }));
  const dataRetirada =
document.getElementById('checkoutRetirada')?.value || new
Date().toISOString().split('T')[0];
  const dataDevolucao =
document.getElementById('checkoutDevolucao')?.value || new
Date(Date.now()+86400000*2).toISOString().split('T')[0];
  try {
    const response = await fetch(`${API_URL}/alugueis`, {
      method: 'POST',
      headers: authHeaders(),
      body: JSON.stringify({ itens, dataRetirada, dataDevolucao })
    });
    if (!response.ok) throw new Error('Erro ao criar reserva');
  }

```

```

    const data = await response.json();
    alert(`Reserva criada! Sinal de R$ ${data.sinalPago.toFixed(2)}
(30%)`);
    localStorage.removeItem('cart');
    updateCartCount();
    showView('home');
  } catch (error) {
    alert('Erro: ' + error.message);
  }
}

```

No HTML, você precisa adicionar campos de data no checkout (ex: `<input type="date" id="checkoutRetirada">` e `<input type="date" id="checkoutDevolucao">`).

☑️ Resumo do que você precisa fazer

1. **Criar o banco de dados** no SQL Server com os scripts fornecidos.
2. **Configurar o backend** (Node.js) com as dependências, ajustar o `.env` com seus dados de conexão.
3. **Rodar o backend** (`npm install` e `npm start`).
4. **Adaptar o frontend** substituindo as funções que usam `localStorage` por chamadas `fetch` (como mostrado acima). Você pode manter o carrinho no `localStorage` para não perder itens ao recarregar a página, mas ao finalizar envia para a API.
5. **Testar** com os usuários padrão (será necessário criar um admin manualmente no banco ou via rota de registro – você pode inserir um admin direto no SQL com senha hash usando `bcrypt`).

Se precisar de ajuda com algum passo específico ou de uma versão mais completa do frontend já integrado, posso fornecer o arquivo HTML completo ajustado. Basta pedir!